Rapport d’avancement

DEV4.5

BEN FRAJ Arwa
BOUNNI LOUBELO Benicia
2026

Table des matières

★ Introduction.....p02

★ Description des fonctionnalités de l'application.....p03

★ Présentation de la structure du programme.....p09

★ Explication de l'algorithme de fin de partie.....p13

★ Conclusions personnelles.....p14

Introduction

Dans ce projet, il faut réaliser une application Android en Java inspirée du jeu de puzzle Chuzzle. Le principe est de manipuler une grille de 6×6 cases en faisant glisser des lignes ou des colonnes avec une rotation circulaire, afin de former des séries d'au moins trois éléments identiques.

L'idée est de reproduire ce fonctionnement tout en respectant un certain nombre de contraintes techniques : gestion des déplacements, détection des combinaisons, système de score et génération aléatoire contrôlée via une graine.

Pour que ça marche, on s'est servi de plusieurs mécanismes :

- une grille générée aléatoirement à partir d'une graine, ce qui permet de rejouer exactement la même partie ;
- un système de détection qui vérifie après chaque glissement si une série de trois cases identiques ou plus est formée, horizontalement ou verticalement ;
- un système de score qui récompense les longues séries et les combos en cascade ;
- des fonctionnalités supplémentaires comme un mode hard avec des cases spéciales (cases mystère, cases persistantes) et un système de verrouillage qui bloque certaines lignes et colonnes au fil du jeu.

L'application est organisée selon le patron MVC : le modèle gère l'état de la grille, le contrôleur contient toute la logique de jeu, et la vue Android s'occupe uniquement de l'affichage et des gestes du joueur.

Description des fonctionnalités de l'application

Menu principal

L'écran d'accueil propose plusieurs options :

- Lancer une nouvelle partie en mode normal ou mode hard.
- Rejouer une partie précédente en entrant sa graine manuellement.
- Consulter les règles du jeu.

Partie classique

L'écran de jeu affiche la grille 6×6. Le joueur interagit en faisant glisser son doigt horizontalement ou verticalement pour décaler une ligne ou une colonne. Un glissement n'est accepté que s'il produit un alignement d'au moins trois cases identiques. On compte des cases normales et des cases verrouillées apparaissant progressivement qui affichent une icône de cadenas et bloquent tout glissement de leur ligne et de leur colonne jusqu'à leur élimination..

En haut de l'écran, deux indicateurs sont mis à jour en temps réel :

- Le score, qui s'incrémente à chaque série éliminée, avec un bonus multiplié par le nombre de combos en cascade.
- Le nombre de coups joués.

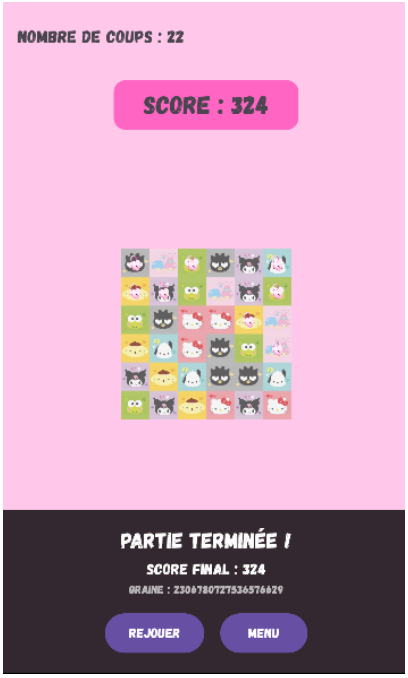
Les cases spéciales (mode hard uniquement) sont représentées visuellement :

- Les cases mystère affichent une icône spéciale de point d'interrogation et doivent être alignées pour être éliminées.
- Les cases persistantes affichent leur couleur avec une icône superposée de punaise ; elles nécessitent 3 alignements pour être éliminées.

La détection de fin de partie s'effectue automatiquement après chaque coup : l'algorithme simule tous les glissements possibles sur toutes les lignes et colonnes non verrouillées. Si aucun ne produit d'alignement, la partie est déclarée terminée.

Un bandeau de fin de partie apparaît alors en bas de l'écran. Il affiche :

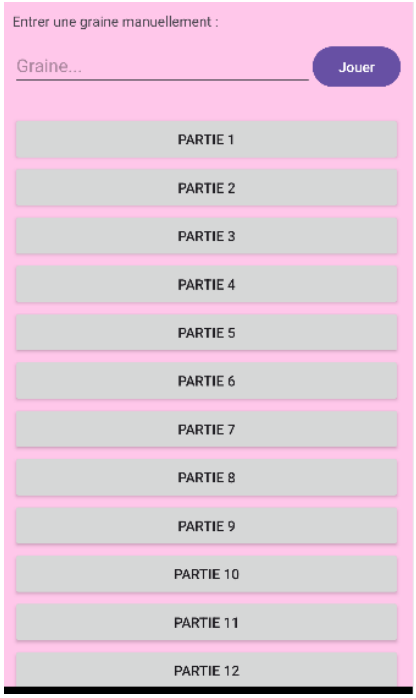
- Le score final.
- La graine de la partie, permettant de la rejouer à l'identique.
- Un bouton pour rejouer immédiatement.
- Un bouton pour retourner au menu.



Partie destinée

L'écran permet de lancer une partie à partir d'une graine spécifique.

Le joueur peut saisir la graine manuellement ou la sélectionner parmi les graines des parties précédemment terminées, qui sont sauvegardées localement à chaque fin de partie. C'est ainsi que nous avons interprété l'énoncé concernant la fonctionnalité de "partie destinée" : la sauvegarde ne constitue pas un historique à part entière, mais sert uniquement à proposer des graines déjà jouées lors du lancement d'une nouvelle partie destinée.



Règles du jeu

L'écran affiche les règles de l'application :

- Le principe du glissement circulaire des lignes et colonnes.
- La condition pour former une série valide (3 cases identiques ou plus).
- L'explication des cases spéciales disponibles en mode hard : cases mystère et cases persistantes.
- Le système de score et les bonus de combo.

RÈGLES DU JEU

🎯 BUT DU JEU

Marquez un maximum de points en alignant des cases de même couleur, jusqu'à ce qu'aucun coup ne soit plus possible.

🟦 LA GRILLE

La grille est composée de 6×6 cases, chacune d'une couleur parmi 7. Au départ, aucune couleur n'apparaît plus de deux fois de suite sur une même ligne ou colonne.

👆 LES COUPS

Faites glisser une ligne vers la gauche/droite ou une colonne vers le haut/bas. Les cases qui sortent d'un côté réapparaissent de l'autre.

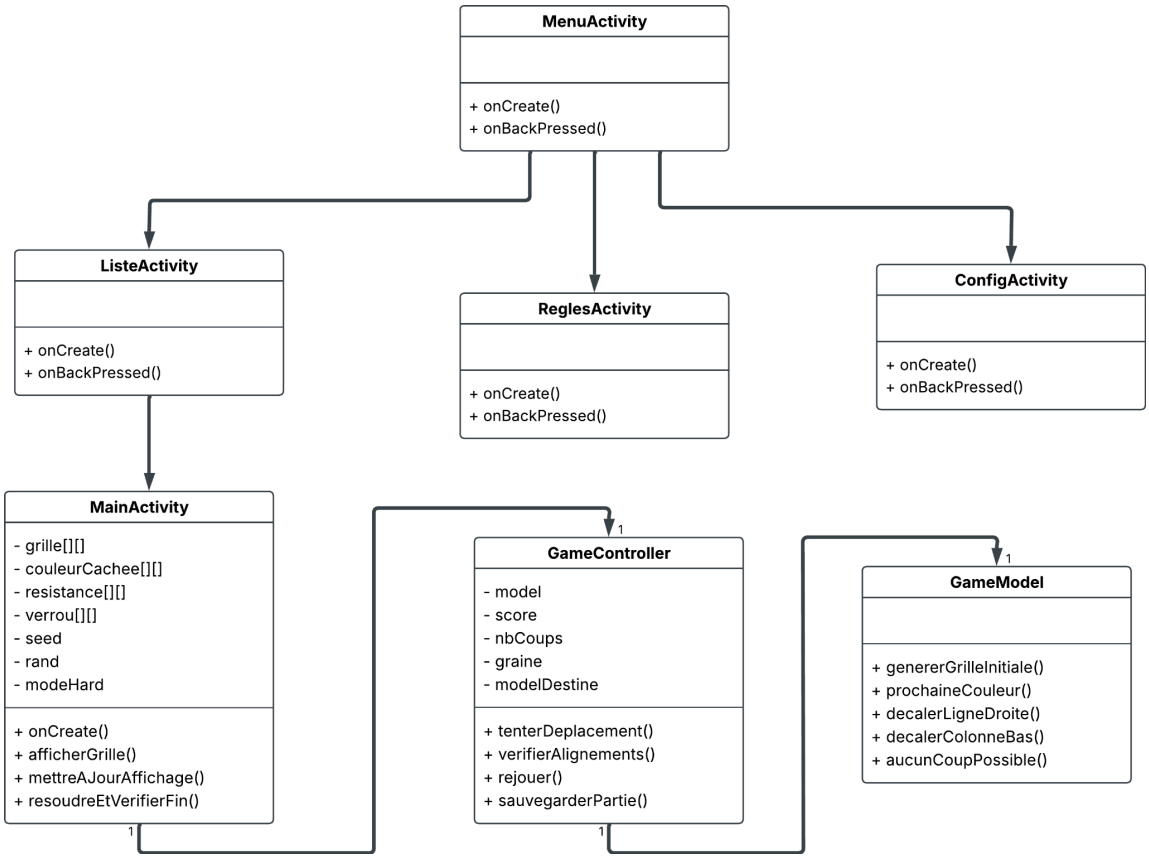
Attention : un coup n'est valide que s'il crée un alignement d'au moins 3 cases de même couleur. Sinon, la grille revient à son état précédent.

💥 LES ALIGNEMENTS

Les cases alignées (3 ou plus) sont retirées. Les cases au-dessus tombent pour combler les vides, et de nouvelles cases apparaissent en haut. Si cela crée de nouveaux alignements, ils sont résolus automatiquement (combo).

🔒 LES VERROUS

Présentation de la structure du programme



Le projet repose sur une séparation claire entre la logique de jeu et la gestion de l'interface Android, organisée selon le patron MVC.

Le code s'articule autour de trois classes principales, chacune ayant un rôle bien délimité.

GameModel - l’état du jeu

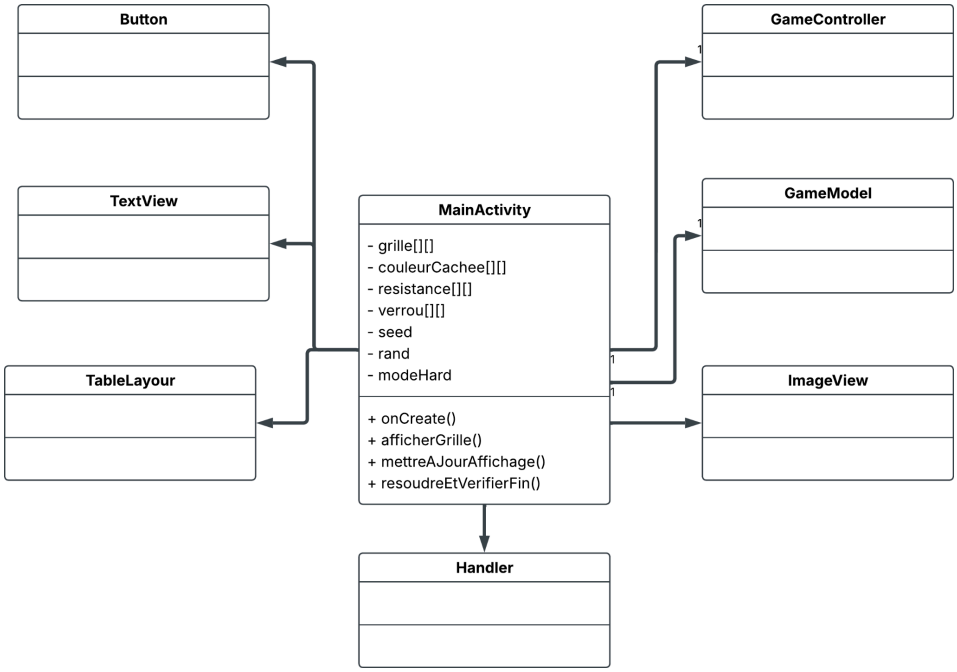
Cette classe a pour rôle de stocker et de manipuler l'état brut de la grille. Elle contient la grille 6×6, les données associées à chaque case (couleur cachée, résistance, verrou), la graine aléatoire et le mode de jeu. Elle expose des opérations purement mécaniques : décaler une ligne ou une colonne, générer de nouvelles cases, détecter des alignements. Elle ne prend aucune décision sur la validité d'un coup — ce n'est pas son rôle.

GameController - la logique du jeu

À partir de l'état fourni par le GameModel, cette classe orchestre toutes les décisions du jeu. Elle valide ou annule un coup selon qu'il produit un alignement, résout les combos en cascade, calcule le score, détecte la fin de partie et gère la sauvegarde des graines. C'est elle qui connaît les règles : le GameModel ne sait pas ce qu'est un coup invalide, le GameController si.

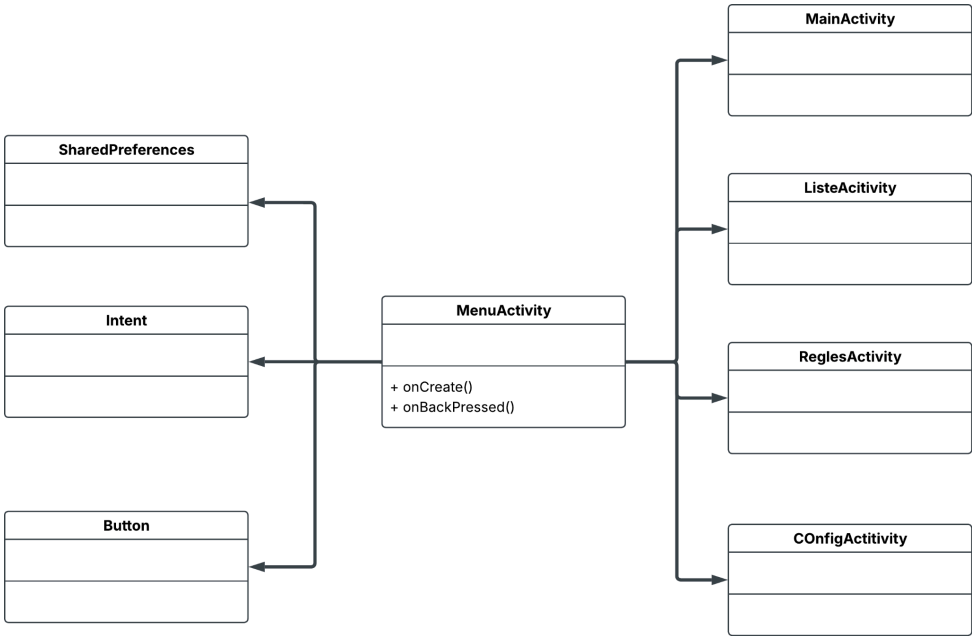
MainActivity - l'affichage et la captation des interactions

Cette classe ne contient aucune logique de jeu. Elle capte les gestes tactiles du joueur, les traduit en intention de déplacement et délègue la décision au GameController. Elle redessine ensuite la grille en lisant l'état du GameModel. Elle gère également le cycle de vie Android afin que la partie survive aux rotations d'écran.



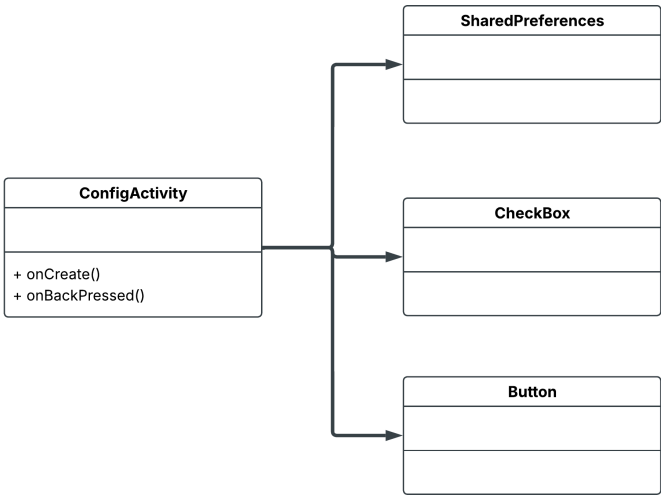
MenuActivity - la navigation

Cette activité sert de point d'entrée de l'application. Elle propose les différentes options de démarrage (partie normale, partie destinée, règles, paramètres) et redirige vers l'activité correspondante via des Intents. Elle lit également les SharedPreferences pour récupérer le mode hard configuré et le transmettre à MainActivity au lancement d'une partie.



ConfigActivity - la configuration du mode hard

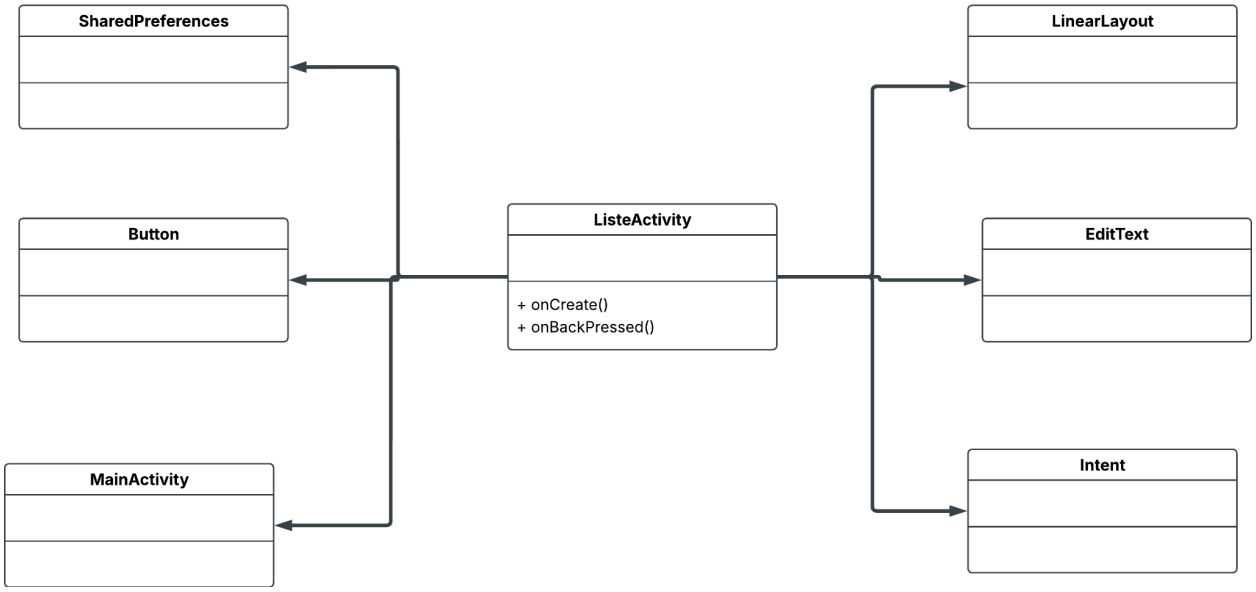
Cette activité permet au joueur d'activer ou de désactiver le mode hard via une case à cocher. Le choix est sauvegardé dans les SharedPreferences sous la clé `modeHard` et relu par MenuActivity à chaque lancement de partie. Elle ne connaît ni GameModel ni GameController.



ListeActivity - la partie destinée

Cette activité permet de lancer une partie à partir d'une graine spécifique. Le joueur peut soit saisir une graine manuellement, soit choisir parmi les graines des parties

précédemment terminées, qui sont lues depuis les SharedPreferences. Une fois la graine sélectionnée, elle est transmise à MainActivity via un Intent.



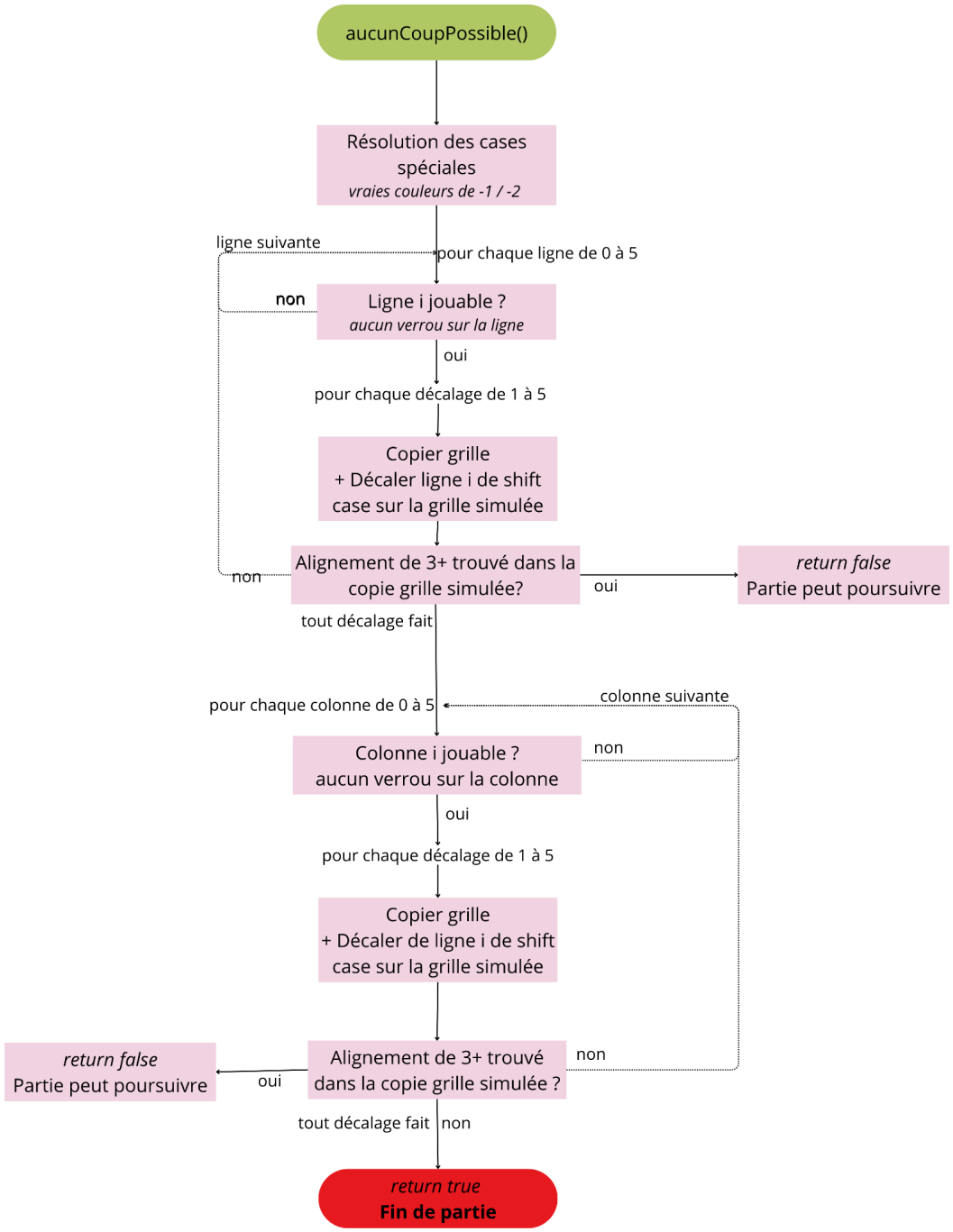
Les relations entre les classes

MainActivity possède un GameController et ne lui parle que via tenterDeplacement(). GameController possède un GameModel et est le seul à le modifier. MainActivity peut lire le GameModel via getModel() uniquement pour l'affichage, sans jamais le modifier directement. Les activités annexes (MenuActivity, ConfigActivity, ListeActivity, ReglesActivity) n'interagissent pas avec le MVC : elles se limitent à la navigation, à la configuration et à la transmission de paramètres via Intents et SharedPreferences.

Explication de l'algorithme de fin de partie

L'algorithme se trouve dans aucunCoupPossible() dans GameModel. Voici comment il fonctionne :

L'idée centrale est simple : plutôt que de regarder la grille actuelle, l'algorithme simule tous les coups possibles et vérifie si l'un d'eux crée un alignement. Si aucun ne le fait, la partie est finie.



Schéma

de l’algorithme:

L'algorithme fonctionne en trois temps :

1. Préparer

Il construit d'abord une copie "résolue" de la grille où les cases spéciales (-1 mystère, -2 persistante) sont remplacées par leur vraie couleur cachée. Cela permet de simuler sans avoir à gérer les cas spéciaux à chaque test.

2. Simuler tous les coups

Pour chaque ligne non verrouillée, il essaie les 5 décalages possibles (1 à 5 cases vers la droite, ce qui couvre aussi les décalages gauche par symétrie circulaire). Pour chaque colonne non verrouillée, pareil avec les décalages verticaux. Chaque simulation travaille sur une copie indépendante de la grille ainsi la vraie grille n'est jamais touchée.

3. Décider

Dès qu'une simulation produit un alignement de 3+, l'algorithme retourne false immédiatement, soit un coup existe, la partie continue. Si toutes les simulations échouent, il retourne true, soit aucun coup n'est jouable, c'est la fin de partie.

Conclusions personnelles

Arwa

Ce projet a été une expérience très enrichissante, notamment grâce au travail de groupe qui a été, selon moi, l’un des points les plus positifs. Nous avons su échanger efficacement tout au long du développement, en prenant le temps de discuter des choix à faire et en nous répartissant les tâches de manière équilibrée. Le fait de travailler à tour de rôle sur certaines parties nous a permis de rester organisés et d’avoir une vision claire de l’avancement du projet et des objectifs à atteindre.

J’ai également beaucoup apprécié le côté créatif du projet. Le fait de concevoir un jeu mobile nous a permis d’apporter notre propre touche personnelle, ce qui rend notre travail unique. Cela a rendu le développement plus motivant et même amusant par moments, tout en restant formateur.

L’analyse du sujet a été une étape assez longue au départ, mais elle s’est révélée essentielle. Prendre le temps de bien comprendre les règles et les attentes nous a permis d’éviter des erreurs par la suite et de progresser plus efficacement dans le développement.

Enfin, ce projet m’a permis de monter en compétences, aussi bien sur le plan technique que sur le plan organisationnel. J’ai pu approfondir mes connaissances en programmation et en développement Android, mais aussi améliorer ma capacité à travailler en équipe, à gérer un projet et à m’adapter aux contraintes. C’est donc une expérience globale très positive que je retiens de cette SAE.

Benicia

Ce projet, réalisé en binôme, a été une expérience très différente du précédent travail en solo. Travailler à deux nous a permis d'échanger régulièrement sur nos choix techniques, de nous répartir les tâches et de confronter nos approches lorsque nous rencontrions un problème. Cette dynamique a rendu le développement plus fluide sur certains aspects, même si elle a aussi nécessité une coordination constante pour garder une vision cohérente du projet.

Nous avons été impliqués dans tous les aspects du développement : la logique de jeu, la gestion des cas spéciaux, le respect du patron MVC, ainsi que la rédaction du rapport. Le fait de devoir structurer le code de manière à ce que chacun puisse intervenir dessus nous a naturellement poussés à soigner l'organisation et la lisibilité.

Si nous devions recommencer, nous aurions mis en place des tests plus systématiques, notamment pour la détection de fin de partie et la gestion des verrous, qui se sont révélés plus délicats que prévu.

En revanche, le temps consacré à la conception et à la compréhension de l'énoncé nous a permis d'aborder le codage de manière plus structurée. Même si certaines fonctionnalités ont demandé plusieurs itérations avant de fonctionner correctement, nous sommes satisfaits du résultat final et des compétences acquises tout au long de ce projet.